



廣東工業大學

GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32F103C8T6 的智能手表设计》

姓 名： 郭浩哲

学 号： 3124009589

专 业： 微电子科学与工程

班 级： 二班

队 名： 克比楞组

队 友： 周瑜亮、叶锦前

指导老师： 周贤中



集成电路学院

2026 年 7 月 11 日

摘要

本文介绍了嵌入式系统课程设计项目——基于 STM32F103C8T6 的智能手表设计。项目以 STM32F103C8T6 最小系统板为核心主控，采用 FreeRTOS 抢占式多任务内核调度，集成 OLED 显示、MPU6050 六轴传感器、EC11 旋转编码器与 JDY-31 蓝牙模块，实现了时间显示、运动计步、多级菜单交互、蓝牙无线数据同步与熄屏唤醒等功能。软件采用软件 I2C 与硬件 I2C1 双总线物理隔离设计，复用片内 RTC 外设走时，并通过片内 Flash 实现步数掉电持久化。在 VSCode+CMake+arm-none-eabi-gcc 工具链下完成固件开发，并在面包板原型上完成功能验证与性能测试。项目展示了 STM32 微控制器与 FreeRTOS 实时操作系统在可穿戴设备中的综合应用价值。

关键词： STM32F103C8T6；FreeRTOS；智能手表；计步算法；蓝牙通信

目录

1	项目简介	1
2	项目设计目标	2
3	项目设计与实现	3
3.1	硬件设计	3
3.1.1	系统总体架构	3
3.1.2	主控制器模块	3
3.1.3	显示模块	3
3.1.4	运动传感器模块	3
3.1.5	旋转编码器模块	3
3.1.6	蓝牙通信模块	4
3.1.7	电源与开机模块	4
3.2	软件设计	4
3.2.1	系统软件架构	4
3.2.2	FreeRTOS 任务划分	4
3.2.3	任务间同步机制	5
3.2.4	系统启动流程	5
3.3	代码介绍	5
3.3.1	片内 RTC 走时实现	5
3.3.2	内部 Flash 步数持久化	6
3.3.3	计步滤波算法	6
3.3.4	任务调度实现	7
3.4	技术实现细节	7
3.4.1	I2C 双总线物理隔离	7
3.4.2	片内 RTC 后备域走时	8
3.4.3	Flash 擦写寿命优化	8
3.4.4	抬腕唤醒 EXTI 设计	8
4	功能展示	9
4.1	功能描述	9

4.1.1	基础功能	9
4.1.2	扩展功能	9
4.2	演示结果	9
4.2.1	计步算法验证	9
4.2.2	走时与熄屏唤醒	10
4.2.3	蓝牙双向同步	10
4.2.4	资源占用	10
5	物料与成本	11
5.1	物料清单	11
5.2	项目成本	11
5.3	损耗与损坏	11
6	个人贡献	12
6.1	报告撰写	12
6.2	编码工作	12
6.3	硬件连接	13
6.4	资本投入	13
6.5	开发中遇到的问题及解决方案	13
6.5.1	抬腕唤不醒（三轮排查）	13
6.5.2	开机偶发卡死且不熄屏	14
6.5.3	计步过于灵敏	14
6.5.4	蓝牙接收断链	14
7	项目成果与反思	15
7.1	成果展示	15
7.2	项目评估	15
7.2.1	成功之处	15
7.2.2	不足之处	16
7.3	个人反思	16
A	设计图纸	17
B	代码清单	17

参考文献

18

1 项目简介

随着可穿戴设备与物联网技术的快速发展，智能手表作为人机交互的重要终端，融合了实时操作系统、多传感器采集、低功耗电源管理与无线通信等嵌入式核心技术。传统电子表功能单一、交互方式有限，难以满足现代用户对运动监测、信息显示与无线数据同步的多样化需求。将嵌入式实时操作系统与多传感器集成应用于可穿戴平台，能够有效提升产品的智能化水平与用户体验。

STM32F103C8T6 微控制器基于 ARM Cortex-M3 内核，具备 72MHz 主频、64KB Flash、20KB SRAM 以及丰富的定时器与通信外设，结合 FreeRTOS 抢占式实时内核，为多任务并发调度提供了理想平台。本项目依托嵌入式系统课程设计，设计并实现一款基于 STM32F103C8T6 的智能手表，将实时操作系统、I2C 总线通信、传感器融合、片内 RTC 走时、内部 Flash 持久化与低功耗管理等知识综合应用于可穿戴场景。

本设计的主要功能包括：

1. **时间显示：**复用片内 RTC 外设与板载 32.768kHz 晶振（LSE）走时，OLED 显示时间页面，熄屏期间 RTC 位于独立后备域持续计时，唤醒后时间不丢失。
2. **运动计步：**MPU6050 六轴传感器采集加速度，动态阈值波峰检测算法解算步数，步数历史支持片内 Flash 掉电持久化。
3. **多级菜单：**EC11 旋转编码器配合 TIM2 硬件正交解码，实现多级菜单切换、参数调节与按键确认。
4. **蓝牙同步：**JDY-31 经典蓝牙模块经 USART2 与 PC 端 PyQt5 上位机双向同步时间、步数与传感器数据。
5. **熄屏与唤醒：**10s 无操作自动熄屏（仅关 OLED），编码器按键或 MPU6050 抬腕运动中唤醒亮屏。

本项目不仅涵盖 FreeRTOS 任务调度、I2C 双总线物理隔离、片内 RTC 走时、内部 Flash 持久化、计步滤波算法与蓝牙异步通信等核心知识点，还涉及硬件电路设计（电源管理、一键开机）与系统级联调等实践技能，是对嵌入式系统课程所学知识的综合应用与实践。

2 项目设计目标

随着可穿戴设备的发展，传统电子表在交互方式、运动监测与数据同步方面的局限性日益突出。基于嵌入式微控制器与实时操作系统的智能手表，能够通过多任务调度、传感器融合与无线通信实现更丰富的功能。本项目结合嵌入式系统课程所学知识，以 STM32F103C8T6 为核心，基于 FreeRTOS 实时操作系统设计一款智能手表，旨在将理论知识与工程实践相结合，探索嵌入式系统在可穿戴设备中的应用潜力。

本项目旨在完成课程大纲全部基本要求并攻克扩展要求 1、2、4，主要设计目标包括：

- **硬件电路设计：**完成以 STM32F103C8T6 为核心的手表硬件系统，集成 1.3 寸 OLED 显示、MPU6050 六轴传感器、EC11 旋转编码器、JDY-31 蓝牙模块与锂电池电源系统，并搭建一键开机电路。
- **FreeRTOS 多任务管理：**移植 FreeRTOS 抢占式内核，划分电源管理、传感器采集、UI 渲染、蓝牙通信四个任务，实现时间显示与传感器数据读取的并发调度，任务间通过消息队列与互斥锁安全通信。
- **OLED 界面显示：**采用软件模拟 I2C 驱动 OLED，实现多页面菜单（时间页、传感器页、计步页、校时页），并完成界面可视化重构。
- **旋转编码器多级菜单（扩展 1）：**利用 TIM2 硬件正交解码 EC11 信号，以结构体数组状态机实现多级菜单切换与选择。
- **蓝牙上位机同步（扩展 2）：**JDY-31 经 USART2 与 PC 端 PyQt5 图形化上位机双向同步，实现运动数据远程读取与时间校准。
- **运动计步与数据记录（扩展 4）：**基于加速度的动态阈值波峰检测计步算法，步数 RAM 缓存与片内 Flash 末页掉电持久化。

预期成果为一套功能完整、可在面包板原型上稳定运行的智能手表软硬件系统，覆盖课程基本要求与扩展要求 1、2、4，并通过现场验收。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统总体架构

系统采用“单主控 + 外围功能模块 + 实时操作系统”架构，以 STM32F103C8T6 为核心，通过 I2C、USART、TIM 等总线连接外设，走时基准由片内 RTC 独立维护，供电采用一键开机拓扑与 LDO 线性降压。为提高总线可靠性，OLED 采用软件模拟 I2C，MPU6050 采用硬件 I2C1，两路总线物理隔离。

3.1.2 主控制器模块

选用 STM32F103C8T6 最小系统板作为主控，主要特性包括：

- 72MHz ARM Cortex-M3 内核，64KB Flash，20KB SRAM
- 片内 RTC 外设（32 位秒计数器），复用板载 32.768kHz 晶振作为 LSE 时钟源
- 3 个通用定时器（TIM2/TIM3/TIM4），硬件 I2C1，USART2
- 丰富的 GPIO 与外部中断（EXTI）资源

3.1.3 显示模块

采用 1.3 寸 OLED 128×64 屏，通过**软件模拟 I2C**（GPIO 位翻转）驱动，引脚分配为 PB10（SCL）/PB11（SDA）。采用软件 I2C 可彻底规避 STM32F1 硬件 I2C 外设已知的总线死锁缺陷，且 OLED 刷新对时序不敏感，适合 GPIO 位翻转方式。

3.1.4 运动传感器模块

采用 MPU6050 六轴传感器，通过**硬件 I2C1**（PB6/PB7）驱动，保证采样的实时吞吐。MPU6050 的运动检测中断 INT 引脚接 PA4（EXTI4），用于抬腕唤醒。

3.1.5 旋转编码器模块

采用 EC11 旋转编码器，A/B 相接 PA0/PA1（TIM2 CH1/CH2），利用定时器编码器模式硬件正交解码；微动按键接 PB12（EXTI），用于菜单确认与唤醒。

3.1.6 蓝牙通信模块

采用 JDY-31 经典蓝牙（SPP）模块，通过 USART2（PA2=TX/PA3=RX）与主控通信，出厂波特率 9600。模块独立供电，与 PC 端 PyQt5 上位机双向同步。

3.1.7 电源与开机模块

供电采用 3.7V 锂电池 + TP4056 充电模块 + XC6206 3.3V LDO 稳压。一键开机电路采用 SI2301（PMOS）+ S8050（NPN）软自锁拓扑：按键上电后主控拉高 PA8（PWR_HOLD）维持 NPN 导通实现自锁，长按可软关机（关机前落盘步数）。

3.2 软件设计

3.2.1 系统软件架构

软件采用分层设计：

1. **硬件抽象层（HAL）**：STM32CubeMX 生成的外设初始化代码与 FreeRTOS 中间件集成。
2. **驱动层（BSP）**：软件 I2C（OLED）、硬件 I2C（MPU6050）、USART、TIM 编码器、RTC、内部 Flash 读写模块。
3. **应用层（App）**：FreeRTOS 任务、计步滤波算法、多级菜单状态机、蓝牙通信协议、步数持久化模块。

3.2.2 FreeRTOS 任务划分

基于功能需求，将系统划分为四个抢占式任务，优先级从高到低依次为电源、传感器、UI、通信：

表 1 FreeRTOS 任务划分

任务名	优先级	触发方式	核心职责
Task_Power	4（最高）	500ms 周期 + 事件	熄屏/亮屏管理、唤醒事件处理
Task_Sensor	3（高）	20ms 周期	MPU6050 采样、计步运算、低频落盘
Task_UI	2（中）	事件驱动	菜单状态机、OLED 渲染、读 RTC
Task_Comm	1（低）	数据队列触发	蓝牙收发、指令解析、状态推送

Task_Power 虽为最高优先级，但采用“周期唤醒 + 事件标志”而非死循环轮询，唤醒后即让出 CPU，不会饿死其他任务。

3.2.3 任务间同步机制

跨任务共享数据由互斥锁（Mutex）保护：UI 任务与传感器任务共享的 `g_ui` 结构体由互斥锁 `s_mtx` 保护，避免读写撕裂；UART 接收到完整一行命令时通过二值信号量 `s_sem_uart` 通知 `Task_Comm` 处理。由于 OLED 与 MPU6050 分别走独立总线，每条总线仅被单一任务访问，不存在共享总线竞争，无需为 I2C 增设互斥锁。

3.2.4 系统启动流程

1. 上电复位，`main()` 在 `SystemClock_Config` 之前拉高 PA8 实现软自锁开机。
2. `HAL_Init` 与 72MHz 时钟树配置，初始化软件 I2C、硬件 I2C1、USART2、TIM2。
3. 片内 RTC 以 LSE 走时初始化（LSE 缺失时回退 LSI）。
4. 从内部 Flash 末页读回步数历史，初始化计步模块。
5. FreeRTOS 内核初始化，创建互斥锁、信号量与四大任务，开启调度器。

3.3 代码介绍

3.3.1 片内 RTC 走时实现

`bsp_rtc.c` 封装了 RTC 时间的设置与读取。关键点在于 STM32F1 的 RTC 读取时序：调用 `HAL_RTC_GetTime` 后必须紧跟 `HAL_RTC_GetDate` 以解锁 shadow 寄存器，否则计数器会被锁住不再更新。

```
void RTC_GetTime(ClockTime_t *out)
{
    RTC_TimeTypeDef t = {0};
    RTC_DateTypeDef d = {0};
    HAL_RTC_GetTime(&hrtc, &t, RTC_FORMAT_BIN);
    HAL_RTC_GetDate(&hrtc, &d, RTC_FORMAT_BIN); /* 必须连读以解锁 */
    out->hour = t.Hours;
    out->min  = t.Minutes;
    out->sec  = t.Seconds;
}
```

跨天判定通过简化的天序号（年 $\times 372$ + 月 $\times 31$ + 日）实现，用于触发当日步数清零。

3.3.2 内部 Flash 步数持久化

bsp_flash.c 将步数历史写入 STM32F103C8T6 片内 Flash 末页(地址 0x0800FC00)。使用魔数 "WAT1" (0x57415431) 标识有效数据, STM32F1 仅支持半字 (16bit) 编程, 故按半字循环写入结构体:

```
uint8_t Flash_SaveSteps(const StepStore_t *in)
{
    StepStore_t buf = *in;
    buf.magic = STORE_MAGIC;
    HAL_FLASH_Unlock();
    /* 擦除末页 */
    FLASH_EraseInitTypeDef erase = {0};
    erase.TypeErase    = FLASH_TYPEERASE_PAGES;
    erase.PageAddress  = FLASH_STORE_ADDR;
    erase.NbPages       = 1;
    HAL_FLASHEx_Erase(&erase, &page_err);
    /* 按半字写入整个结构体 */
    const uint16_t *src = (const uint16_t *)&buf;
    for (uint32_t i = 0; i < halfwords; i++) {
        HAL_FLASH_Program(FLASH_TYPEPROGRAM_HALFWORD, addr, src[i]);
        addr += 2;
    }
    HAL_FLASH_Lock();
    return 0;
}
```

为延长 Flash 擦写寿命 (约 1 万次), 采用 “仅步数变化时擦写 + 熄屏前强制落盘” 策略, 避免每周期无意义擦写。

3.3.3 计步滤波算法

pedometer.c 采用纯整数运算 (适合 Cortex-M3 无 FPU), 算法流程为:

1. 计算三轴合加速度 $A = \sqrt{A_x^2 + A_y^2 + A_z^2}$ (用整数平方根 `isqrt32` 实现);
2. 长度 N 滑动窗口算术平均, 滤除高频噪声;
3. 动态阈值: 过去 50 点的 $(\max + \min)/2$ 作为步态波峰基准线;
4. 当前加速度上穿基准线, 且幅度超过门限、距上次计步 $> 200\text{ms}$, 则计 1 步。

$$A = \sqrt{A_x^2 + A_y^2 + A_z^2}$$

$$\text{threshold} = \frac{\max_{50}(\bar{A}) + \min_{50}(\bar{A})}{2}$$

其中 $\bar{A}$ 为滑动平均后的合加速度。

3.3.4 任务调度实现

`app_tasks.c` 实现四大任务。Task_Sensor 以 20ms 周期读取 MPU6050、执行计步、低频落盘; Task_Power 以 500ms 周期检查空闲超时; Task_UI 响应编码器事件并渲染; Task_Comm 处理蓝牙收发。熄屏采用“仅关 OLED、MCU 全速运行”方案 (不进 STOP 模式):

```
static void enter_idle(void)
{
    persist_steps(1);          /* 熄屏前强制落盘 */
    OLED_DisplayOff();
    s_screen_off = 1;
}
```

3.4 技术实现细节

3.4.1 I2C 双总线物理隔离

OLED 走软件 I2C (独立 GPIO PB10/PB11), MPU6050 走硬件 I2C1 (PB6/PB7), 两路总线物理隔离, 每条总线仅被单一任务访问, 从根本上消除了总线访问竞争, 也规避了 STM32F1 硬件 I2C 的总线死锁缺陷。

3.4.2 片内 RTC 后备域走时

RTC 位于独立后备域，熄屏期间（仅关 OLED，MCU 全速运行）RTC 持续计时，从根本上解决了软件 SysTick 计时在深度休眠后停摆的问题，唤醒后时间不丢失。

3.4.3 Flash 擦写寿命优化

记录最近一次落盘的步数 `s_saved_total`，仅在步数较上次有变化时才真正擦写 Flash，把擦写频率从“每 60s 一次”降到“仅步数变动时”，避免几天内写穿末页。

3.4.4 抬腕唤醒 EXTI 设计

MPU6050 运动检测中断 INT 接 PA4(EXTI4)。熄屏时运动中断触发 `mark_activity()`，`Task_Power` 下个周期检测到活动时间被刷新即调用 `exit_idle()` 亮屏，最多 500ms 延迟。亮屏时运动中断仅计数不唤醒，避免佩戴微动导致永不熄屏。

4 功能展示

4.1 功能描述

本系统实现了基于 STM32F103C8T6 与 FreeRTOS 的智能手表功能，包括基础功能与扩展功能两大部分。

4.1.1 基础功能

- **片内 RTC 走时：**复用片内 RTC 外设与板载 32.768kHz 晶振（LSE）走时，OLED 时间页显示时分秒，熄屏期间 RTC 持续计时。
- **六轴传感采集：**MPU6050 经硬件 I2C1 每 20ms 采样一次加速度与陀螺仪数据。
- **FreeRTOS 多任务：**四任务抢占式调度，任务间通过互斥锁与信号量安全通信。
- **多页面显示：**OLED 实现时间页、传感器页、计步页、校时页四页菜单，界面可视化重构（反白标题栏、大字号、进度条）。
- **独立电源系统：**3.7V 锂电池供电，Type-C 充电，一键开机软自锁电路。

4.1.2 扩展功能

- **旋转编码器多级菜单（扩展 1）：**EC11 + TIM2 硬件正交解码，结构体状态机实现四页切换、数值调节与按键确认。
- **蓝牙上位机同步（扩展 2）：**JDY-31 经 USART2 与 PyQt5 上位机双向同步，实时显示时间、步数、加速度，支持校时与请求同步。
- **运动计步与数据记录（扩展 4）：**动态阈值波峰检测计步，步数 RAM 缓存与片内 Flash 末页掉电持久化。
- **熄屏与抬腕唤醒：**10s 无操作熄屏（仅关 OLED），编码器按键或 MPU6050 抬腕运动中唤醒。

4.2 演示结果

4.2.1 计步算法验证

采用合成步态信号离线验证：

- 静止 5s: 计 0 步, 无误触发;
- 走路 10s@2Hz: 计 19 步 (预期约 20 步), 误差 < 5%, 逻辑正确。

真机佩戴时阈值偏灵敏, 静止或轻微动作时仍会计步、步数偏多, 精度为本项目已知不足 (详见第 7 章)。

4.2.2 走时与熄屏唤醒

- RTC 以 LSE 走时, 熄屏期间持续累计, 唤醒后时间不丢失。
- 静置 10s 准时熄屏; 手持微动同样 10s 熄屏。
- 抬腕或按键在 500ms 内亮屏。

4.2.3 蓝牙双向同步

- 上位机每秒收到一行状态 T:hh:mm:ss S:total/today A:ax,ay,az。
- 上位机发送 SET 12 34 56, 表盘时间立即变更为 12:34:56。
- 连续收发 1 分钟不断流, 熄屏后蓝牙仍可收发。

4.2.4 资源占用

arm-none-eabi-gcc 编译无警告, RAM 占用 75.0%, Flash 占用 74.2% (界面重构后), 余量充足。

完整的真机联调故障排查过程与验证步骤见随仓库提交的 Docs/故障排查记录.md。

5 物料与成本

5.1 物料清单

本项目主控核心板由课程提供，其余元器件通过线上配单采购。物料清单如表2所示。

表 2 系统物料清单

物料名称	规格	数量	单价 (元)	用途
主控核心板	STM32F103C8T6 最小系统板	1	0.00	核心控制（学校提供）
OLED 屏	1.3 寸 I2C	1	14.50	显示输出
旋转编码器	EC11 带微动开关	1	3.50	菜单交互
蓝牙模块	JDY-31 带底板	1	6.50	无线同步
六轴传感器	MPU6050	1	6.80	姿态/计步采集
锂电与电源包	3.7V 锂电池 + TP4056 + XC6206	1 套	11.00	供电与充电管理
一键开机元件	SI2301 + S8050 + 按键	1 套	2.50	开关机控制
基础辅料	面包板 + 杜邦线 + 阻容	1 套	7.00	原型搭建
合计			51.80	

5.2 项目成本

本项目采用面包板模块化互联，不做 PCB 打样，成本控制在较低水平：

- **材料成本：**约 51.80 元（见表2）。
- **工具与设备：**ST-Link 调试器、万用表等使用课程实验室资源，无额外采购成本。
- **开发软件：**VSCode + CMake + arm-none-eabi-gcc 工具链均为开源免费。
- **总成本：**约 51.80 元。

5.3 损耗与损坏

项目实施过程中产生的损耗较少：

- 杜邦线损耗若干根（已含在辅料包内，无额外损失）。
- 无核心元器件烧毁或损坏。
- **总损耗：**可忽略，已含在材料成本内。

6 个人贡献

6.1 报告撰写

本人负责项目中与内核、算法相关文档内容的撰写与维护，主要包括：

- 撰写《系统设计文档》中软件部分详细设计，包括 FreeRTOS 任务划分、计步滤波算法描述、RTC 走时与 Flash 持久化方案。
- 编写《故障排查记录》中抬腕唤醒、卡死不熄屏等缺陷的根因分析与代码修复点说明。
- 参与中期检查报告中算法与内核部分的进度汇报。

项目总体文档由队友叶锦前主笔整合，本人负责内核与算法相关章节的技术内容。

6.2 编码工作

本人承担项目“内核与算法”方向的全部编码工作，主要包括：

- **FreeRTOS 任务框架：**移植 FreeRTOS 抢占式内核，划分 Power/Sensor/UI/Comm 四任务并配置优先级，设计任务间互斥锁（保护共享结构体 `g_ui`）与二值信号量（UART 整行通知）的同步机制。
- **片内 RTC 走时：**封装 `bsp_rtc.c`，实现基于片内 RTC（LSE）的时分秒读取，处理“GetTime 后必须 GetDate 解锁 shadow 寄存器”的关键时序，并实现跨天判定（天序号比较触发当日步数清零）。
- **内部 Flash 持久化：**实现 `bsp_flash.c`，将步数历史写入 Flash 末页（0x0800FC00），使用魔数标识有效数据，按半字编程写入结构体；设计“仅步数变化时擦写 + 熄屏前强制落盘”策略延长 Flash 寿命。
- **计步滤波算法：**实现 `pedometer.c`，采用纯整数运算（整数平方根、滑动平均、动态阈值波峰检测），适合 Cortex-M3 无 FPU 场景；算法参数集中定义于 `app_config.h` 便于标定。
- **Task_Sensor 与 Task_Power 实现：**编写传感器任务的 20ms 周期采样、计步、低频落盘逻辑，以及电源任务的周期 + 事件式熄屏管理。

6.3 硬件连接

本人主要负责软件方向，硬件电路搭建与电源调试由队友周瑜亮负责。本人参与了以下与软件相关的硬件联调工作：

- 参与 MPU6050 硬件 I2C1 接口的 bring-up 与传感器数据读取验证。
- 参与一键开机电路 PA8 (PWR_HOLD) 软自锁逻辑的联调，定位并修复了“无代码拉高 PA8 导致脱离 ST-Link 后松键断电”的严重缺陷。
- 协助 MPU6050 运动中断 INT→PA4 (EXTI4) 接线与抬腕唤醒链路的真机标定。

6.4 资金投入

项目物料由小组成员共同采购、费用均摊，总成本约 51.80 元，本人承担其中约三分之一。

6.5 开发中遇到的问题及解决方案

6.5.1 抬腕唤不醒（三轮排查）

- **问题描述：** STOP 休眠后抬腕无法唤醒，只能靠编码器按键唤醒。
- **排查过程：**
 - 第一轮怀疑 INT 锁存未清除与 DLPF 过滤，改非锁存脉冲模式，真机复测仍唤不醒。
 - 第二轮查数据手册，发现遗漏 SIGNAL_PATH_RESET(0x68)=0x07 信号通路复位，运动检测前置高通滤波器残留旧状态导致 INT 根本不产生；且 $\sim 50\mu\text{s}$ 脉冲太短，EXTI 抓不到边沿。
 - 第三轮定位真正根因：STOP 模式下 Task_Power 抢占 Task_Sensor 的 I2C 事务时存在**总线竞争**（约 20% 概率返回 HAL_BUSY），锁存 INT 线保持高电平，STOP 中无新上升沿，抬腕不唤醒。
- **解决方案：** 移除 STOP 模式，熄屏仅关 OLED、MCU 全速运行，EXTI 始终被 ISR 正常服务；补齐信号通路复位 + 锁存模式 + DLPF 放宽至 1.25Hz。
- **经验：** “按键能醒、抬腕不醒”应第一时间二分到传感器出中断这一环；外设手册里“看似可省”的复位步骤往往是功能不工作的元凶。

6.5.2 开机偶发卡死且不熄屏

- **问题描述：**开机偶发卡在 TIME 页约 10s、不响应、不熄屏。
- **根因：**MPU6050 运动中断每触发即刷新活动计时，佩戴微动导致永不熄屏；PA4 浮空引发 EXTI 中断风暴饿死所有任务；Task_Sensor 优先级高于 UI，硬件 I2C 异常时 HAL 空转 25ms 等超时也会饿死 UI。
- **解决方案：**运动中断改为仅计数、亮屏时不刷新活动计时；PA4 改内部下拉杜绝浮空中断风暴；Task_Sensor 连续 5 次 I2C 失败后退避 200ms 并重置节拍基准。

6.5.3 计步过于灵敏

- **问题描述：**真机佩戴时计步阈值偏灵敏，静止或轻微动作仍会计步，步数偏多。
- **分析：**离线合成步态验证逻辑正确（静止 0 步、走路 19 步 @2Hz），但真机佩戴时动态阈值窗口与幅度门限未充分标定。
- **改进方向：**提高幅度门限 PED_MIN_AMPLITUDE、延长最小步间隔 PED_MIN_STEP_INTERVAL、增大动态阈值窗口 PED_DYN_WINDOW，或引入活动强度判别（静止时抑制计步）。

6.5.4 蓝牙接收断链

- **问题描述：**HAL 的 UART 收发共用一把锁，中断里重挂接收若撞上任务发送持锁返回 BUSY，接收从此断链；ORE/FE 错误后 HAL 不自动恢复。
- **解决方案：**新增 HAL_UART_ErrorCallback 清错并重挂接收；Task_Comm 加 RX 看门狗兜底（断链 100ms 内恢复）。

7 项目成果与反思

7.1 成果展示

本项目成功设计并实现了一套基于 STM32F103C8T6 与 FreeRTOS 的智能手表系统，主要成果包括：

- 完成课程大纲全部基本要求与扩展要求 1、2、4，覆盖“优秀”等级的扩展组合要求。
- 实现 FreeRTOS 抢占式四任务调度，任务划分合理、无死锁。
- 实现片内 RTC（LSE）走时与内部 Flash 步数掉电持久化。
- 实现动态阈值波峰检测计步算法（纯整数运算，适合无 FPU 场景）。
- 实现软件 I2C / 硬件 I2C1 双总线物理隔离，从根本上消除总线竞争。
- 实现抬腕/按键唤醒的熄屏方案，并完成 OLED 界面可视化重构。
- 通过现场验收。

7.2 项目评估

7.2.1 成功之处

- **总线可靠性：**OLED 软件 I2C 与 MPU6050 硬件 I2C1 物理隔离，既规避了 STM32F1 硬件 I2C 死锁缺陷，又消除了共享总线竞争，长时间运行稳定。
- **RTC 走时方案：**复用片内 RTC 后备域走时，熄屏期间持续计时，解决了软件计时在休眠后停摆的问题，且无需额外 RTC 芯片，BOM 成本不变。
- **Flash 寿命优化：**“仅步数变化时擦写 + 熄屏前落盘”策略有效延长 Flash 擦写寿命。
- **抬腕唤醒排查：**经三轮排查定位 STOP 模式 I2C 总线竞争根因并彻底解决，体现了软硬件协同调试能力。
- **工程化配置：**算法参数与引脚分配集中定义于 app_config.h，便于现场标定与维护。

7.2.2 不足之处

- **计步精度不足：**真机阈值标定偏灵敏，静止或轻微动作时仍会计步、步数偏多，是本项目主要不足。离线合成步态验证逻辑正确，但真机佩戴参数未充分标定。
- **无 STOP 低功耗：**因 STOP 模式存在 I2C 总线竞争，移除 STOP 改为仅关 OLED，MCU 全速运行，功耗较 STOP 方案升高（约 30mA vs STOP 约 20 μ A）。低功耗电源管理不在课程考核标准内，此为工程取舍。
- **无 PCB：**采用面包板模块化互联，未做扩展 3（PCB 设计），体积与可靠性不及 PCB 方案。
- **蓝牙平台限制：**JDY-31 为经典蓝牙 SPP，iPhone 不支持，须用 Android 或 PC。

7.3 个人反思

通过本项目，获得了宝贵的嵌入式系统开发经验：

- **RTOS 实践：**深入理解了 FreeRTOS 任务优先级、互斥锁、信号量、周期任务与事件驱动任务的配合，认识到“最高优先级任务忙等会饿死其他任务”的陷阱。
- **软硬件协同调试：**抬腕唤醒问题的三轮排查让我认识到，真机才暴露的缺陷往往根因在软硬件交互（如 I2C 总线竞争），需结合数据手册与示波器/逻辑分析仪思路定位。
- **外设手册细节：**RTC 读取“GetTime 后必须 GetDate”、MPU6050“信号通路复位”等“看似可省”的步骤，恰恰是功能不工作的元凶，体现了仔细研读数据手册的重要性。
- **算法标定：**计步算法离线验证通过但真机不理想，让我认识到算法参数的真机标定与离线验证同等重要。

未来改进方向：

1. 完善计步算法真机标定，引入活动强度判别抑制静止误计步。
2. 通过互斥锁或时序设计解决 STOP 模式下的 I2C 竞争，恢复深度低功耗。
3. 制作 PCB 提升体积与可靠性。
4. 增加心率等传感器扩展功能。

A 设计图纸

完整的硬件框图、软件流程图与引脚分配见随仓库提交的《系统设计文档》：

- 硬件框图 (Mermaid)：Docs/系统设计文档.md
- 软件启动流程图 (Mermaid)：Docs/系统设计文档.md
- 引脚常量定义：Firmware/App/Inc/app_config.h
- CubeMX 工程配置：Firmware/SmartWatch.ioc

B 代码清单

本项目固件源码位于 Firmware/App/Src 目录，上位机位于 Host/。本人负责的内核与算法相关源文件如下：

- app_tasks.c：FreeRTOS 四大任务与系统集成实现
- pedometer.c：计步滤波算法（合加速度 + 滑动平均 + 动态阈值波峰检测）
- bsp_rtc.c：片内 RTC 走时封装
- bsp_flash.c：内部 Flash 步数持久化
- app_config.h：算法参数与引脚分配集中配置

其余源文件（OLED 驱动、菜单、编码器、蓝牙协议、上位机）由队友负责，完整代码见随附仓库。

参考文献

- [1] STMicroelectronics. *STM32F103xx Reference Manual* (RM0008). Rev 21, 2018.
- [2] STMicroelectronics. *STM32F103x8/STM32F103xB Datasheet*. Rev 17, 2015.
- [3] Richard Barry. *Mastering the FreeRTOS Real Time Kernel*. Real Time Engineers Ltd., 2020.
- [4] InvenSense. *MPU-6000/MPU-6050 Product Specification* (PS-MPU-6000A-00). Rev 3.4, 2013.
- [5] 刘火良, 杨森. *STM32 库开发实战指南: 基于野火 STM32 全系列开发板*. 机械工业出版社, 2020.
- [6] FreeRTOS 官方文档. <https://www.freertos.org/>